Prompt Templates & Dynamic Generation

Practical Learning Notes & Interview Preparation Guide

1. Concept Purpose

What problem does this concept solve? Raw string concatenation (e.g. `f"Explain {paper} in {style} style"`) is brittle, error-prone, vulnerable to prompt injection, lacks schema validation, cannot be easily versioned or saved to disk, and fails to handle structured roles (System/Human/AI) cleanly.

Why do we need it? `PromptTemplate` and `ChatPromptTemplate` transform dynamic string construction into first-class LangChain Runnable components. They validate that all expected input variables are supplied, support serialization (JSON/YAML) for decoupled prompt engineering, and integrate seamlessly into execution chains (`prompt | model`).

Where is it used? In parameterized chatbots, few-shot prompt builders, dynamic form interfaces, automated email/code generation systems, and RAG document grounding.

Why did you use it in your implementation? In your `Prompts/` directory, you built a research paper summarization UI using Streamlit. You separated the prompt creation and serialization (`prompt_generator.py`) from UI consumption (`dynamic_prompt_ui.py`), loading the compiled prompt from `template.json` at runtime.

2. Code Structure & Deep Analysis of Your Repository Files

File 1: `Prompts/prompt_generator.py`

```
from langchain_core.prompts import PromptTemplate
from langchain_core.load import dumps
from pathlib import Path

template = PromptTemplate(
    input_variables=["paper_input", "style_input", "length_input"],
    validate_template=True,
    template="""
    Please explain the research paper titled {paper_input}
    in a {style_input} style.
    The explanation should be {length_input} in length.
    """
)

Path("template.json").write_text(dumps(template), encoding="utf-8")
```

Component Breakdown:

- `PromptTemplate`: The core class for string-based prompt templating. It parses placeholders enclosed in single braces `{...}`.
- `input_variables=[...]`: Explicit list of required variable names. When `validate_template=True` is passed, LangChain checks at instantiation whether the variables listed match the exact placeholders inside the template string. If there is a mismatch, it raises a `ValueError` immediately.
- `dumps(template)`: LangChain's serializer from `langchain_core.load`. It converts the LangChain object into a standardized JSON representation with namespace metadata (`{"lc": 1, "type": "constructor", "id": ["langchain", "prompts", "prompt", "PromptTemplate"], ...}`).
- `Path("template.json").write_text(...)`: Writes the serialized prompt to disk, enabling version control and prompt

sharing across services without code changes.

File 2: `Prompts/dynamic_prompt_ui.py`

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import streamlit as st
from langchain_core.prompts import load_prompt

load_dotenv()

model = ChatOpenAI(
    model="openai/gpt-4o-mini",
    base_url="https://openrouter.ai/api/v1"
)

st.header("Research Tool")

paper_input = st.selectbox("Select Research Paper Name", [...])
style_input = st.selectbox("Select Explanation Style", [...])
length_input = st.selectbox("Select Explanation Length", [...])

# Dynamic Prompt Template loaded from disk
template = load_prompt('template.json')

prompts = template.invoke({
    'paper_input': paper_input,
    'style_input': style_input,
    'length_input': length_input
})

if st.button("Generate Explanation"):
    result = model.invoke(prompts)
    st.write(result.content)
```

Component Breakdown:

- `load_prompt('template.json')`: Reads the serialized JSON from disk and reconstructs an executable `PromptTemplate` instance.
- `template.invoke({...})`: Executes the template runnable. It takes a dictionary of variable values, validates them against `input_variables`, performs f-string substitution, and returns a `StringPromptValue`.
- `model.invoke(prompts)`: Receives the `PromptValue` directly. `PromptValue` provides both `to_string()` (for legacy LLMs) and `to_messages()` (for Chat Models).
- `st.button(...)`: Streamlit event trigger. In Streamlit, state re-executes top-to-bottom on every user click, making prompt generation dynamic based on widget values.

File 3: `Prompts/static_prompt_ui.py`

```
# Static comparison: User inputs raw text without a structured template
user_input = st.text_input("Enter your Prompt")
if st.button("Submit"):
    result = model.invoke(user_input)
    st.write(result.content)
```

Comparative Analysis: `static_prompt_ui.py` accepts unconstrained user input. If the user forgets to specify style, length, or context, the model produces inconsistent outputs. In contrast, `dynamic_prompt_ui.py` constrains user choices via UI dropdowns and guarantees high-quality, structured prompts every time.

3. Internal Execution Flow

1. **Template Instantiation & AST Parsing:** When `PromptTemplate` is constructed, LangChain uses Python's `string.Formatter().parse(template)` to extract all variable names enclosed in braces. If `validate_template=True`, it verifies `set(parsed_vars) == set(input_variables)`.
2. **Serialization:** `dumps()` traverses the object attributes and serializes it using LangChain's Serializable protocol (recording module, class name, kwargs, and template format).
3. **Deserialization:** `load_prompt()` parses the JSON/YAML file and dispatches to the registered constructor in `langchain_core`.
4. **Variable Binding (invoke):** When `template.invoke(dict_inputs)` is called:
   - Missing variables raise an `InputValidationError` or KeyError.
   - Extra unexpected variables are ignored or flagged depending on configuration.
   - Python formatting replaces `{paper_input}` with the concrete value.
5. **Output Generation:** Produces a `StringPromptValue` instance wrapping the formatted text.

4. Workflow Diagram

```
+-----------------------------------------------------+ | Streamlit UI: Dropdowns (Paper, Style,
Length) | +-----------------------------------------------------+ | v Input Dictionary:
{'paper_input': '...', ...} +-----------------------------------------------------+ |
load_prompt('template.json') -> PromptTemplate |
+-----------------------------------------------------+ | v .invoke(dictionary)
+-----------------------------------------------------+ | Variable Validation & Substitution |
+-----------------------------------------------------+ | v StringPromptValue
+-----------------------------------------------------+ | ChatOpenAI (gpt-4o-mini via OpenRouter) |
+-----------------------------------------------------+ | v AIMessage
+-----------------------------------------------------+ | Streamlit st.write(result.content) |
+-----------------------------------------------------+
```

5. Recommended Project Architecture

In enterprise systems, prompts are managed like database migrations—stored in dedicated repositories, versioned with semver, and separated by business domain.

```
research_assistant/
|
├── config/
|   └── settings.py
|
├── prompts/
|   ├── __init__.py
|   ├── base.py              # Shared prompt utility helpers
|   ├── templates/           # Stored JSON / YAML templates
|   |   ├── summarizer_v1.json
|   |   ├── summarizer_v2.json
|   |   └── code_explainer.yaml
|   └── prompt_registry.py   # Factory to load and cache templates
|
├── services/
|   └── generation_service.py # Chains: prompt | model | parser
|
├── ui/
|   └── app.py               # Streamlit or FastAPI controller
```

```
│
└── main.py
```

6. Implementation From Scratch (Step-by-Step Exercise)

Objective: Build a Few-Shot Prompt Template that teaches an LLM how to convert natural language queries into SQL statements using few-shot examples.

```
# Reference Solution
from langchain_core.prompts import PromptTemplate, FewShotPromptTemplate

# 1. Define example structure
example_prompt = PromptTemplate(
    input_variables=["question", "sql"],
    template="Question: {question}
SQL: {sql}"
)

# 2. Provide demonstration examples
examples = [
    {
        "question": "Show all employees in the Sales department",
        "sql": "SELECT * FROM employees WHERE department = 'Sales';"
    },
    {
        "question": "Count total products priced above 100",
        "sql": "SELECT COUNT(*) FROM products WHERE price > 100;"
    }
]

# 3. Assemble FewShotPromptTemplate
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix="You are a SQL expert. Translate natural language into valid SQLite queries.",
    suffix="Question: {user_query}
SQL:",
    input_variables=["user_query"]
)

# 4. Invoke
formatted_prompt = few_shot_prompt.invoke({"user_query": "Find the average salary of engineers"})
print(formatted_prompt.to_string())
```

7. Common Errors & Actual Repository Bugs

ERROR 1: Relative Path Ambiguity in Streamlit (From Prompts/dynamic_prompt_ui.py)

Buggy Line: `template = load_prompt('template.json')`

Why It Fails: When running Streamlit from the project root (`d:\langchain`) with `streamlit run Prompts/dynamic_prompt_ui.py`, Python's current working directory is `d:\langchain`. It looks for `template.json` in the root rather than `d:\langchain\Prompts\template.json`. That is why your repository has two copies of `template.json`!

Correct Fix: Resolve paths relative to the current file:

```
from pathlib import Path
```

```
TEMPLATE_PATH = Path(__file__).parent / "template.json"
template = load_prompt(str(TEMPLATE_PATH))
```

ERROR 2: Unmatched Input Variables

If you specify `input_variables=["paper", "style"]`, but your template string contains `{paper}`, `{style}`, and `{length}` with `validate_template=True`, LangChain raises:

```
ValueError: Found input_variables ['length'] that are not in template.
```

Lesson: Always use `from_template()` factory method whenever possible: `PromptTemplate.from_template("Explain {paper} in {style}")`. It automatically extracts and validates variable names without manual duplication.

8. Explain It to Someone Else (Speaking Frameworks)

- **Simple Explanation:** "A PromptTemplate is like a fill-in-the-blanks mad-libs form for an AI. Instead of writing the whole prompt each time, you leave placeholders like `{topic}` and let the program fill them in safely."
- **Technical Explanation:** "PromptTemplates are parameterized Runnable components that compile input dictionaries into concrete `PromptValue` objects through schema validation, AST bracket extraction, and string interpolation, supporting serialization and LCEL pipeline composition."
- **60-Second Interview Explanation:** "In LangChain, hardcoding prompt strings is an anti-pattern. `PromptTemplate` enforces a strict contract between dynamic application inputs and model consumption. In my research paper tool project, I utilized `PromptTemplate` with `validate_template=True` to guarantee schema conformity, serialized the resulting template to JSON via `dumps`, and loaded it dynamically in a Streamlit frontend via `load_prompt`. This decouples prompt tuning from application code."
- **Whiteboard Explanation:** Draw a JSON document `{"template": "...{query}..."}` on the left. Draw a factory `load_prompt()` box in the center. Show runtime dictionary `{"query": "User Question"}` merging into the template. Output is `PromptValue` feeding into `Model`.
- **Real-World Analogy:** HTML templates in web frameworks (Jinja2 or React JSX). You don't build HTML strings manually in route handlers; you render structured templates with variables.

9. Interview Preparation Questions & Answers

Level	Question	Technically Exact Answer
Basic	What is the difference between `PromptTemplate` and `ChatPromptTemplate`?	`PromptTemplate` formats a single text string (used primarily for completion LLMs). `ChatPromptTemplate` formats a list of structured messages (System, Human, AI) with distinct roles for conversational chat models.
Basic	How does `PromptTemplate.from_template()` work?	It is a convenience classmethod that automatically parses the template string for variables enclosed in single braces `{...}`, populates `input_variables` automatically, and configures standard f-string formatting without manual variable declaration.

Intermediate	What is partial formatting in LangChain prompts?	Partial formatting allows you to bind a subset of input variables early, producing a new PromptTemplate that only expects the remaining variables. For example, binding `current_time` or `company_name` once at startup so runtime callers only need to supply `user_question`.
Intermediate	Why serialize prompts to JSON/YAML instead of keeping them in Python files?	Decoupling prompts into JSON/YAML allows non-engineering domain experts and prompt engineers to update, iterate, and A/B test prompts without altering application code or triggering CI/CD code deployments.
Advanced	How do PromptTemplates protect against Prompt Injection attacks?	PromptTemplates enforce clear structural separation between developer instructions and untrusted user input, especially when paired with `ChatPromptTemplate` where user input is strictly quarantined inside `HumanMessage` rather than concatenated into system instructions.
Advanced	What is the return type of `template.invoke()` and why is it important?	It returns a `PromptValue` object (subclassing `Serializable`). `PromptValue` is polymorphic: it has `to_string()` to feed legacy string-based models and `to_messages()` to feed chat models without manual type conversion.